

Contracts, DTOs & validation

Public shape vs EF entities · ProblemDetails

DTO

```
public record CourseResponseDto(  
    Guid Id, string Title,  
    string Code, int MaxCapacity);  
  
// Map in LINQ SQL stays tight
```

Protecting the internal schema

Session goals

By the end of this session, you will:

- Explain why EF entities must not leak to clients
- Design request and response DTOs for TMS resources
- Eliminate circular reference serialization failures
- Return consistent ProblemDetails (RFC 9457)
- Contrast PUT (full replace) vs PATCH (partial)
- Validate with Data Annotations; know when FluentValidation fits

Why not return entities?

- Circular graphs Student ↔ Enrollment breaks JSON serializers
- Security internal fields (notes, hashes) must not cross the wire
- Coupling DBA renames a column and every client breaks

Red flag in review

```
public async Task<List<Student>> GetStudents() entities on the wire.
```

Entity vs DTO vs projection

```
// Internal graph + sensitive field
```

```
public class Enrollment {  
    public Guid Id { get; set; }  
    public Student Student { get; set; }  
    public string InternalComments { get; set; }  
}
```

DTO

```
// External flat contract  
public record EnrollmentDto(  
    Guid Id,  
    string StudentName,  
    DateTime EnrolledAt);
```

LINQ projection

```
return await context.Enrollments  
    .Select(e => new EnrollmentDto(  
        e.Id, e.Student.Name, e.EnrolledAt))  
    .ToListAsync(ct);
```

Why project in LINQ

EF emits SQL for only the columns you need not SELECT * then map in memory. InternalComments never crosses the wire.

Create vs update DTOs

Create

```
// Create  Code once (immutable after enrollments)

public record CreateCourseRequest {

    [Required, MaxLength(200)]

    public required string Title { get; init; }

    [Required, RegularExpression(@"^[A-Z]{3}-\d{3}$",
        ErrorMessage = "Code XXX-000")]

    public required string Code { get; init; }

    [Range(1, 200)]

    public int MaxCapacity { get; init; }
}
```

Update

```
// Update  no Code (FK stability)

public record UpdateCourseRequest {

    [Required, MaxLength(200)]

    public required string Title { get; init; }

    [Range(1, 200)]

    public int MaxCapacity { get; init; }
}
```

PUT vs PATCH

- PUT full replace; client sends every field; omitted fields → null/default risk
- PUT success typically 204 No Content
- PATCH only changed fields (e.g. adjust a grade); saves bandwidth; server applies a delta
- PATCH success 200 OK or 204 No Content (team picks one and documents it)
- TMS emphasis: PUT for admin course forms; understand PATCH for interviews and mobile

Angular validates a grade between 0 and 100 do you still validate on the server?

Data Annotations + ApiController

Validation

```
public record CreateCourseRequest {  
  
    [Required]  
  
    [MaxLength(200)]  
  
    public required string Title { get; init; }  
  
    [Range(1, 200,  
        ErrorMessage = "Capacity 1-200 students")]  
  
    public int MaxCapacity { get; init; }  
}
```

Framework behavior

[ApiController] runs model validation and returns 400 Bad Request with ValidationProblemDetails before your action body executes.

ProblemDetails one error shape

Five endpoints return errors as string, HTML, raw JSON, XML, and plain text. How does Angular write one interceptor?

RFC 9457 (sample)

```
{
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Title": ["The Title field is required."],
    "MaxCapacity": ["Capacity must be 1 to 200 students"]
  }
}
```

```
Program.cs
builder.Services.AddProblemDetails();

// 400 validation vs 409 business  same ProblemDetails family
```

- 400 input / binding failures
- 409 business rule violations (course full, duplicate code)
- One interceptor shape across the whole TMS API

FluentValidation awareness

FluentValidation

```
public class CreateCourseValidator
    : AbstractValidator<CreateCourseRequest>
{
    public CreateCourseValidator() {
        RuleFor(x => x.Title).NotEmpty().MaximumLength(200);

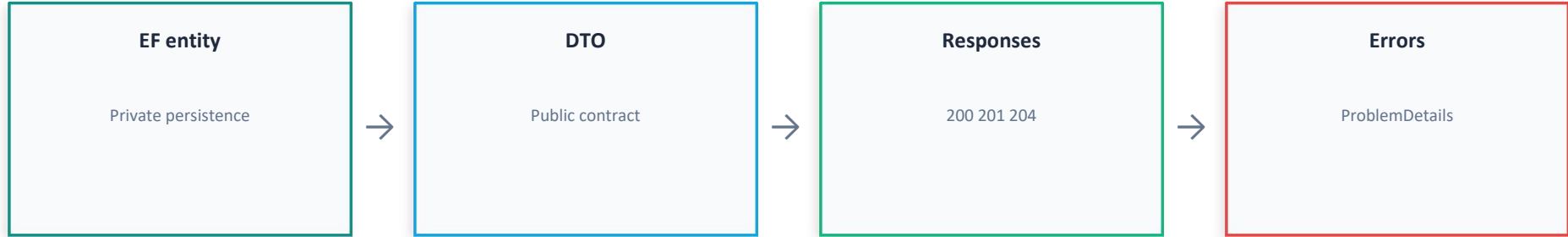
        RuleFor(x => x.Code).NotEmpty()
            .Matches(@"^[A-Z]{3}-\d{3}$");

        RuleFor(x => x.MaxCapacity).InclusiveBetween(1, 200);
    }
}
```

Trade-off

Data Annotations for simple fields; FluentValidation for cross-field, conditional, or shared rules. NuGet: `FluentValidation.AspNetCore`. TMS labs stay annotations-first ≤5 min aside.

Blueprint entity wall



Session summary

Topic	Takeaway
DTOs	Firewall cycles, privacy, decoupling
Mapping	LINQ projection keeps SQL honest
Create vs update	Different writable fields per operation
PUT / PATCH	Full replace vs partial know both
ProblemDetails	One parseable error contract
400 vs 409	Input vs business rule
Server validation	Only layer you can trust always validate on the API

What this looks like in an interview

Why DTOs? Coupling, cycles, and accidental exposure. Pair with: how ProblemDetails keeps Angular error handling boring in a good way.

Next: Session 3 collections & HATEOAS

Mandatory pagination, PagedResponse, filter/count/sort/page order, action filters, and links so clients stop hard-coding URLs.